PROFESSIONAL DEVELOPMENT

Inbound Benefit Deductions Integration: Process Flow

Workday Studio integration that loads a benefits vendor's deduction file and updates employee benefit elections.

At a glance

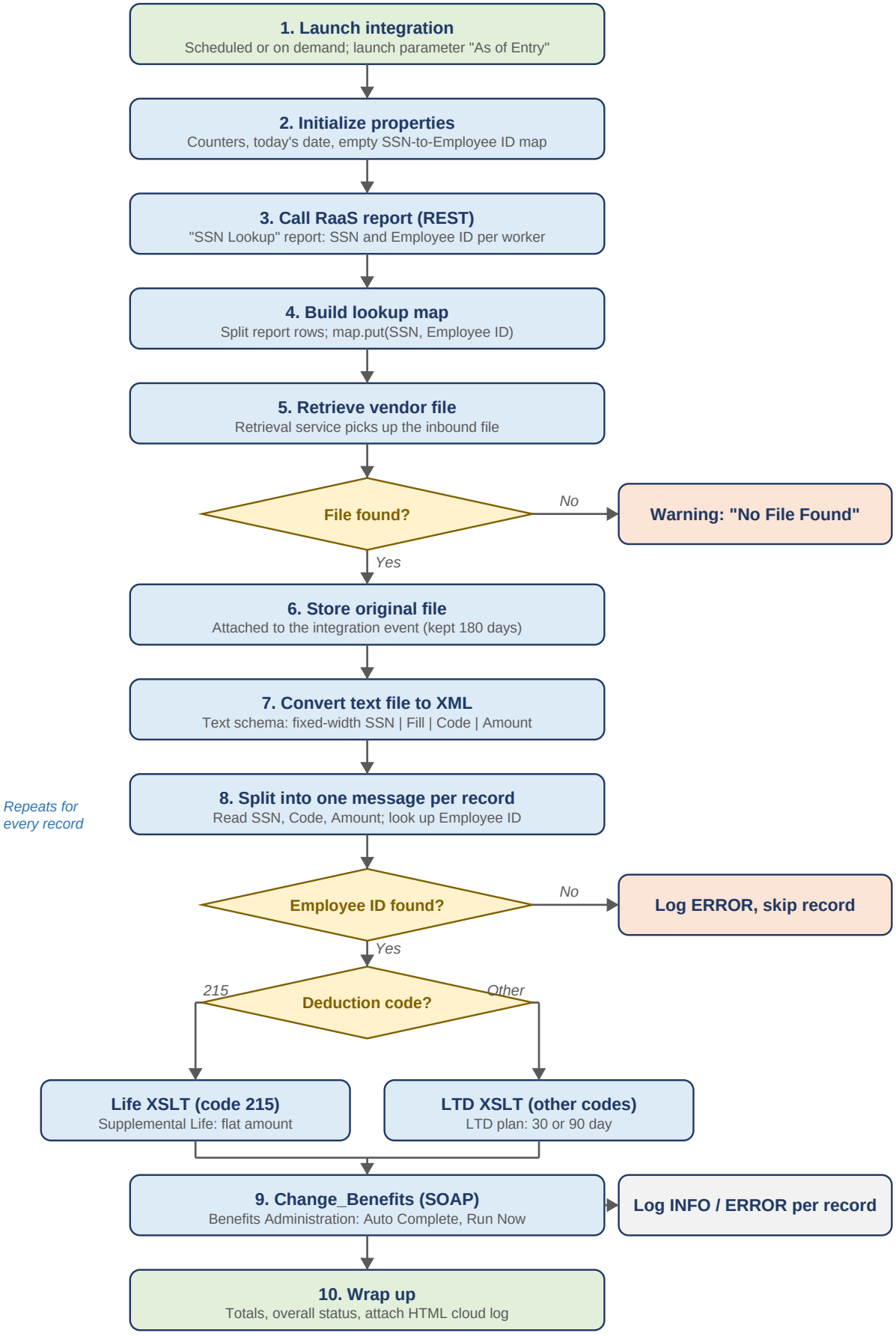
Item	Details
Type	Inbound Workday Studio integration (custom assembly)
Purpose	Read a deduction file from a third-party benefits vendor and start, change or stop Supplemental Life and Long-Term Disability (LTD) elections in Workday
Source	Fixed-width text file, picked up by a Retrieval Service (for example SFTP)
Target	Workday **Change_Benefits** web service (Benefits Administration, SOAP)
Lookup	Custom RaaS report that returns SSN and Employee ID for each worker
Launch	Scheduled or on demand. Launch parameter "As of Entry" (defaults to the current moment)
Outputs	Updated benefit events, original file stored, HTML cloud log attached, summary integration message

Input file layout

Field	Length	Description
SSN	9	Employee Social Security Number; the key used to find the worker
Fill	1	Filler or space (not used)
Code	3	Deduction code: **215** = Supplemental Life, any other code = LTD (**263** = 90-day plan)
Amount	9	Deduction amount in cents (divided by 100 in the XSLT)

The text schema (XSD) turns each line into a *Record* XML element, so the rest of the integration works with XML instead of raw text.

Process flow



Step by step

Step	What happens
1. Launch	The integration system starts, scheduled or on demand. It reads the "As of Entry" launch parameter.
2. Initialize	An Eval step sets properties: record, warning and error counters, today's date, and an empty Java **HashMap** for SSN-to-Employee ID lookups.
3. Call RaaS	**Workday Out REST** calls a custom report that returns every worker's SSN and Employee ID.
4. Build lookup map	A **Splitter** (XPath *Report_Entry*) loops through the report rows and stores each SSN with its Employee ID in the HashMap. Loading the map once avoids calling Workday for every record.
5. Retrieve file	A **Route** checks whether a file was delivered. If not, the integration logs the warning "No File Found" and ends.
6. Store file	The original vendor file is saved and attached to the integration event (kept 180 days) for audit and re-processing.
7. Convert to XML	A **Text Schema** step (text to XML) turns the fixed-width file into XML records.
8. Split and validate	An **XML Stream Splitter** handles one record at a time. It reads SSN, Code and Amount, increments the counter and looks up the Employee ID in the map. A **Validate** step fails the record if no Employee ID is found; the error is logged and the integration moves on to the next record.
9. Route and transform	A **Route** on the deduction code sends the record to the Life XSLT (code 215) or the LTD XSLT (other codes). Each XSLT builds a **Change_Benefits_Request**.
10. Call Workday	**Workday Out SOAP** sends the request to Benefits Administration. Success is logged as INFO ("Record n updated"); failures are logged as ERROR.
11. Wrap up	The integration counts warnings and errors, sets the overall status (INFO, WARNING or ERROR), posts a summary message ("n records processed") and attaches the HTML cloud log.

Business rules in the XSLT

Benefit	Condition	Request sent to Workday
Supplemental Life (code 215)	Amount is not 1 cent	Benefit event *Life*; elect the Supplemental Life additional benefits plan; flat contribution = amount / 100; coverage and deductions start today
Supplemental Life (code 215)	Amount = 1 cent	Same event with **no elections**: treated as "end the benefit"
LTD (other codes)	Amount > 0	Benefit event *LTD*; elect the LTD insurance plan (90-day if code 263, otherwise 30-day), coverage 100% of salary
LTD (other codes)	Amount = 0	Same event with **no elections**: ends the coverage

Every request uses **Auto Complete = true** (no approvals), **Run Now = true** and **Discard On Exit Validation Error = true** (failed events are discarded, not left in progress).

Error handling and logging

Level	Mechanism
Global	A global error handler sends unexpected failures to a CRITICAL integration message.
Per step	Each mediation step has a **Send Error** handler with a clear message, such as "Error calling RaaS report" or "Error transforming inbound file".
Per record	Record-level problems (missing Employee ID, XSLT or SOAP failure) go to a shared **CloudLogInfo** sub-routine. It logs the record number and details and updates the warning and error counts, so one bad record does not stop the whole file.
Audit	The original file and the HTML cloud log are both attached to the integration event.

Studio components used

Workday In, Async Mediation, Eval (MVEL), Route (MVEL strategy), Splitter (XPath and XML Stream), Text Schema, XSLT Plus, Validate Expression, Workday Out REST (RaaS), Workday Out SOAP, Store, Cloud Log, Local In / Local Out sub-routines, Send Error, Put Integration Message.

Interview talking points

Topic	What to say
Design choice	Loading SSN-to-Employee ID into a HashMap once means one report call instead of one call per record, which scales well for large files.
Resilience	Errors are handled per record; failures are logged and counted while the remaining records keep processing.
Security	SSN is used only as a lookup key. Updates in Workday use the Employee ID, and the lookup report should be restricted to the integration system user.
Auditability	Original file and cloud log are stored with the event; the final status shows the warning and error counts.
Possible improvements	Fix the code check in the LTD XSLT (it compares *file.code* instead of *$file.code*, so it always selects the 30-day plan); use number() when comparing amounts; replace the "1 cent = end" convention with an explicit action code; align web service versions (v43 endpoint, v44 request).